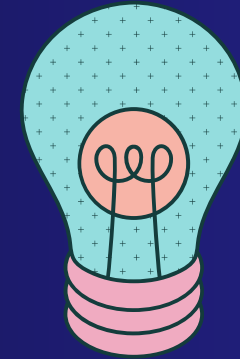
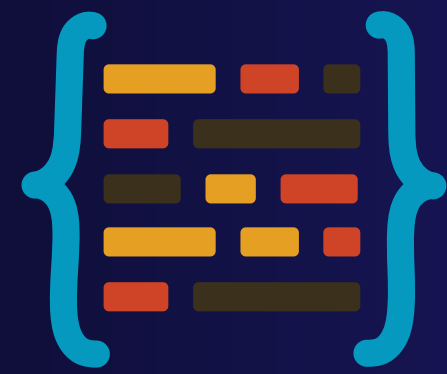


Java Collection Framework



By <Sariful>



www.linkedin.com/in/sarifuli/



www.youtube.com/CodenboxAutomationLab



www.codenboxAutomationlab.com



Table Contents



➞ Introduction 01

➞ Hierarchy of Collection Framework 02

➞ List Interface 03

➞ ArrayList 04

➞ LinkedList 05

06 Set Interface ➞

07 HashSet ➞

08 LinkedHashSet ➞

09 Queue Interface ➞

10 Map Interface & HashMap Class ➞

Introduction



- The Collection in java is a framework/API that provides an architecture to store and manipulate the group of objects.
- It refer to java.util package
- Java Collections can be used for searching, sorting, inserting, manipulating, and deleting data.
- The size is infinite. It can contain multiple types of data.
- provides many interfaces as: (Set, List, Queue) and classes as:(ArrarList, LinkedList, HashSet, LinkedHashSet, TreeSet etc.)

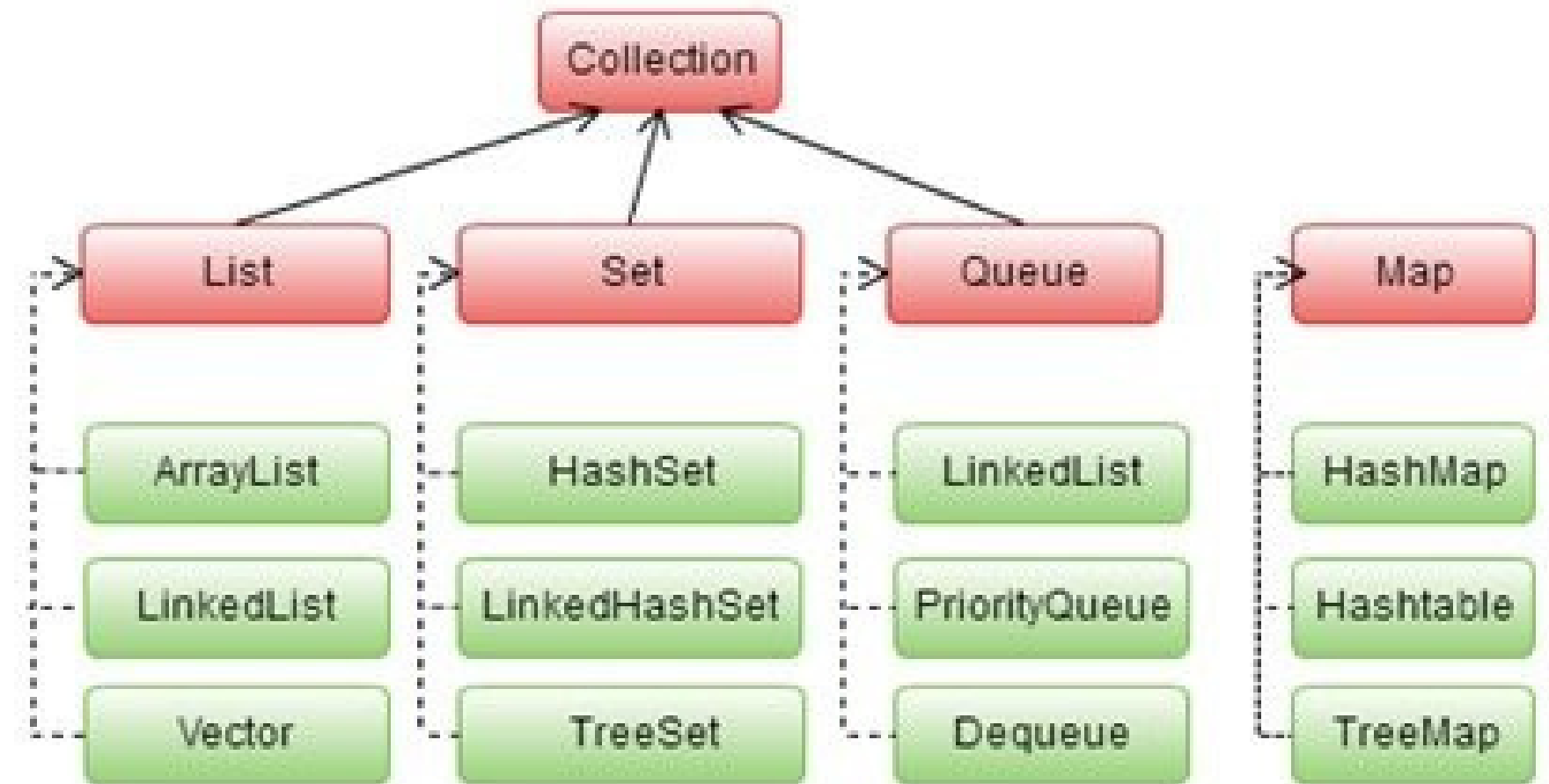


[linkedin.com/in/sarifuli/](https://www.linkedin.com/in/sarifuli/)

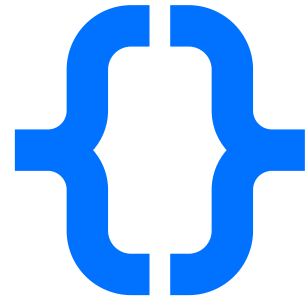
Hierarchy of Collection Framework



- Collection Interface: Is the root of the collection hierarchy.
 - **List**: Ordered collection with duplicate elements (e.g., ArrayList, LinkedList).
 - **Set**: Unordered collection with unique elements (e.g., HashSet, TreeSet).
 - **Queue**: Represents a queue, often used for processing tasks (e.g., PriorityQueue, LinkedList).
- **Map** Interface: Represents a collection of key-value pairs (e.g., HashMap, TreeMap).

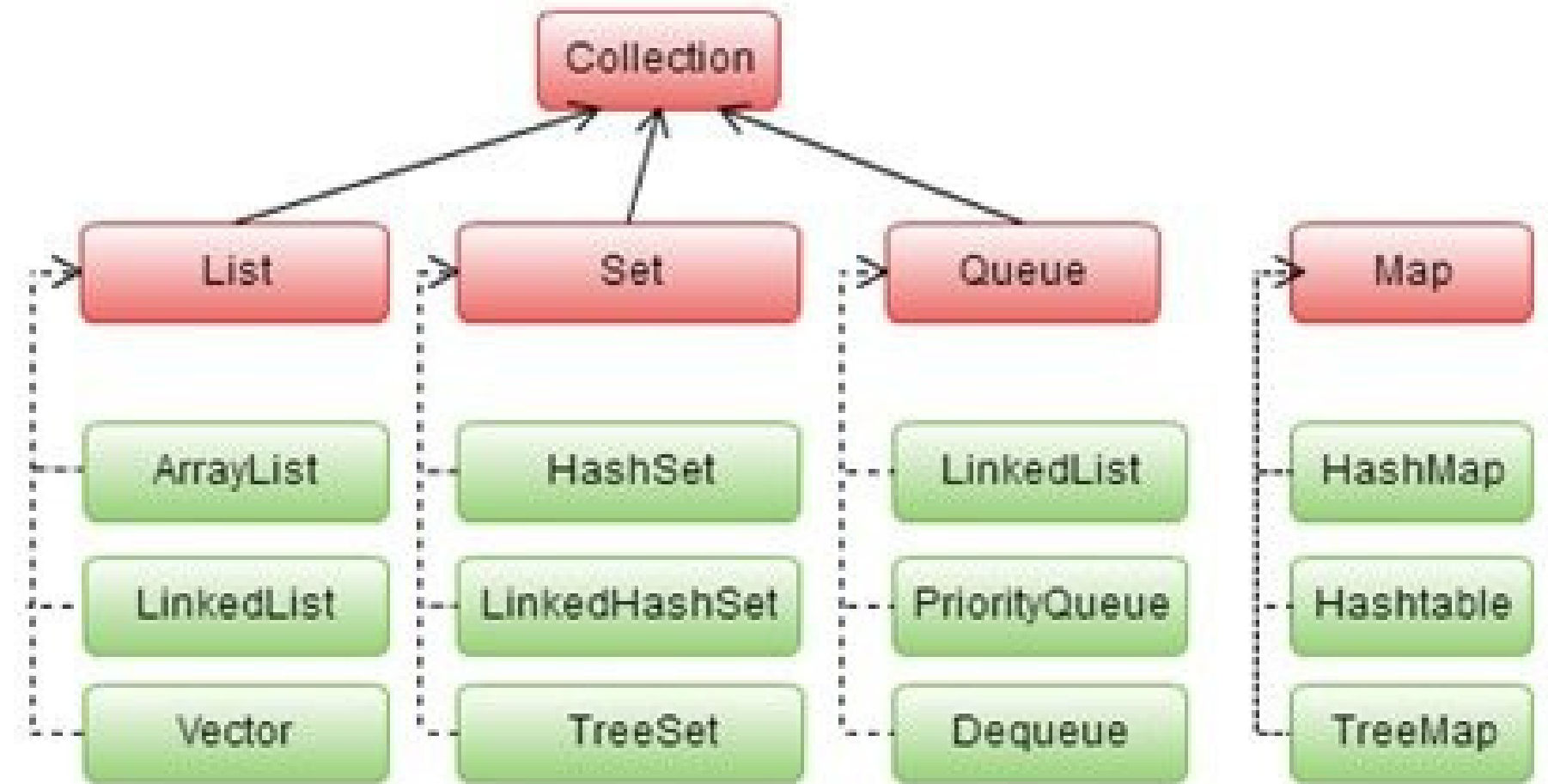


List Interface



- The List interface is part of the java.util package and represents an ordered collection, also known as a sequence. The order follows an indexing system. It may contain duplicate elements. Below classes implement List interface:

- ArrayList
- LinkedList
- Vector

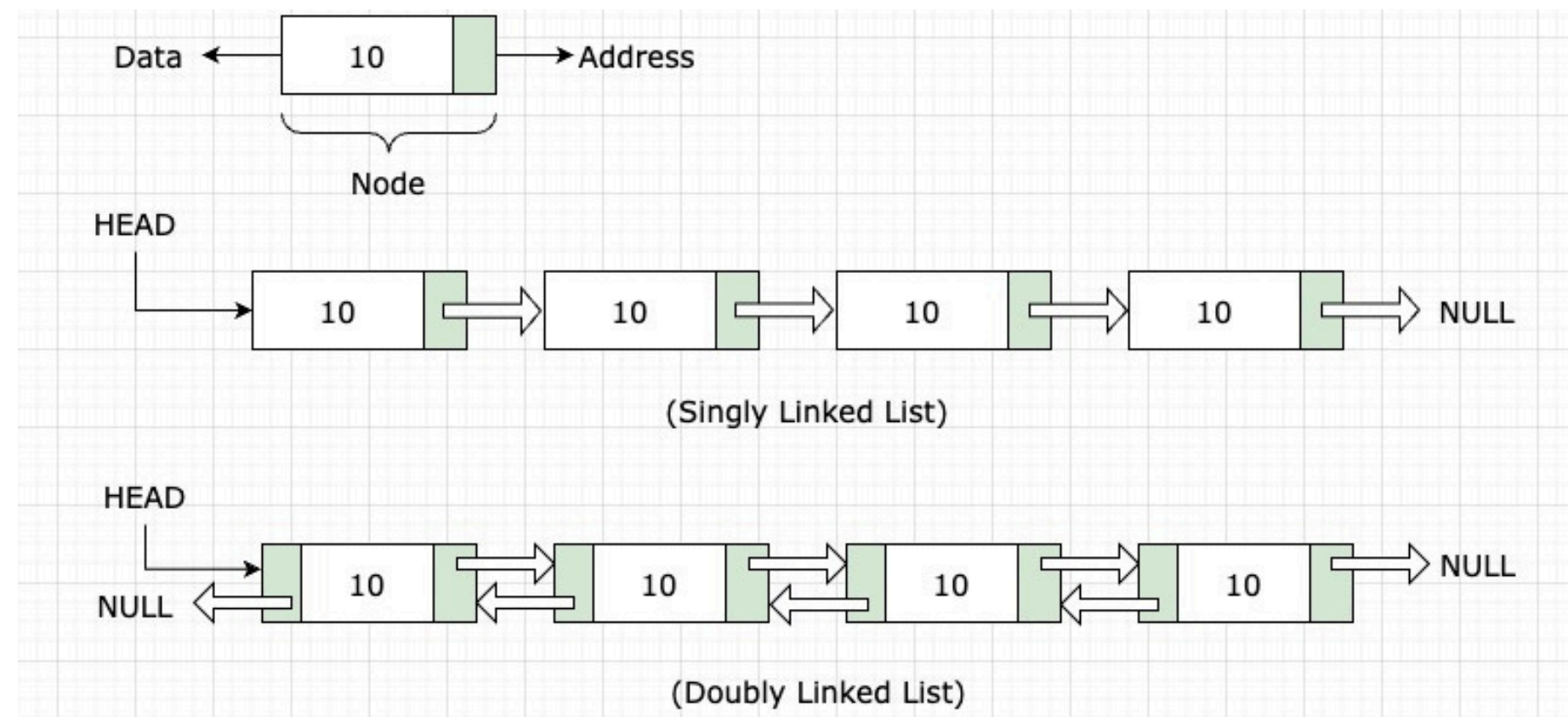


ArrayList

- The list is implemented by the ArrayList class.
- The ArrayList class is a resizable array which can be found in java.util package.
- The difference between regular array and an ArrayList is that the size of an regular array cannot be modified, whereas we can add, remove elements from an ArrayList whenever we want. It can have duplicate values with different types of data.
- ArrayList maintains insertion order.
- In this we don't need to mention the length of the ArrayList, instead of this, it's length increases as new objects are added and shrinks when objects are removed from it.
- ArrayList never stores the 'primitive values'.
- All values/elements stored in the ArrayList are as object. So we create an object of ArrayList class and store 'Integer' for int, 'Boolean' for boolean, 'Character' for char, 'Double' for double data using Wrapper class concept.
- Useful methods in ArrayList are: add(); get(); set(); remove(); size(); clear(); contains(); index of();
- Parameterization of ArrayList: is possible, means that we are allowed to restrict the ArrayList to store only 'same' type of data. Example: `ArrayList<Integer> myList= new ArrayList<Integer>()`

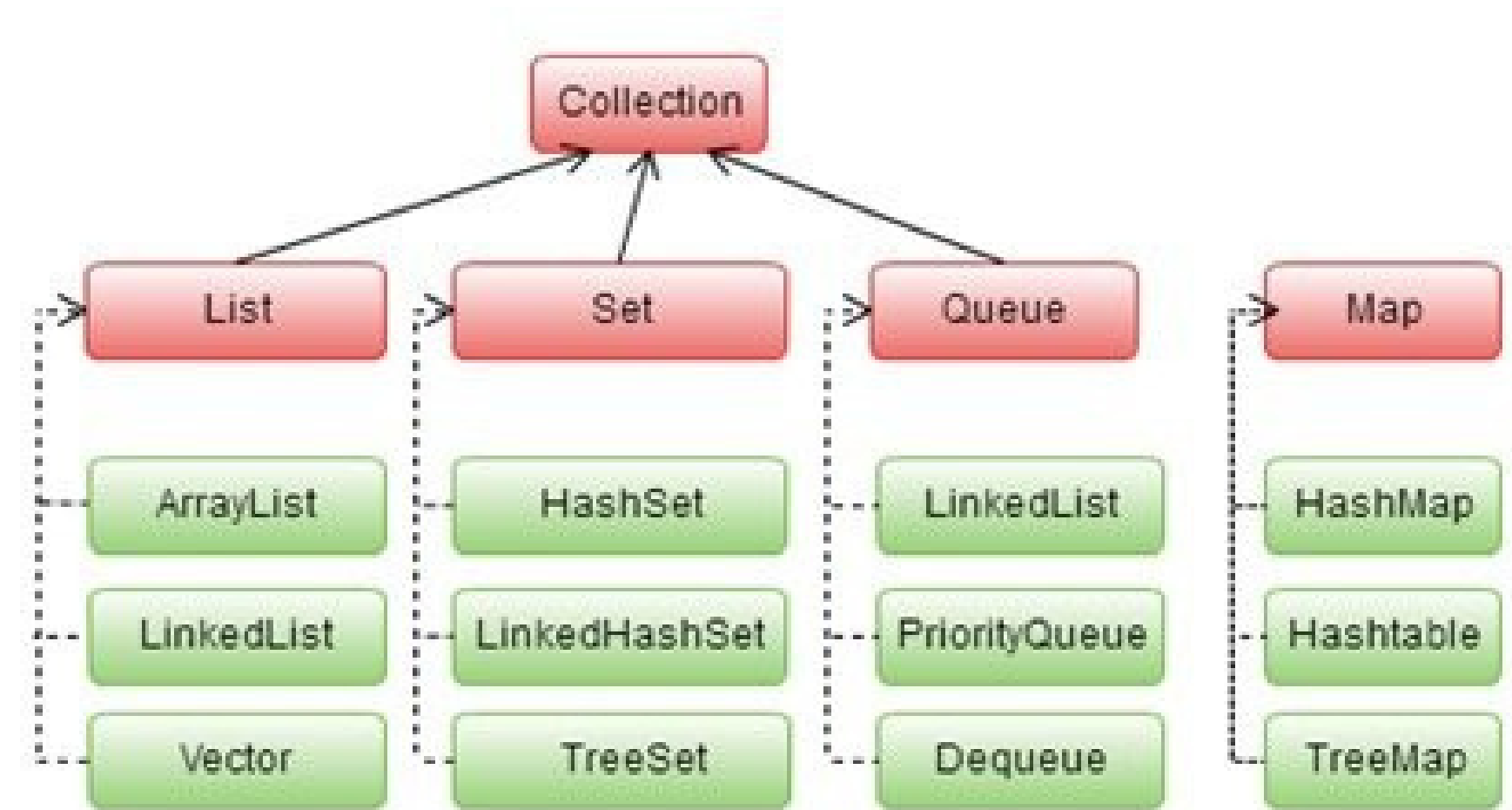
LinkedList

- The LinkedList class is almost identical to the ArrayList.
- Internally, the elements are stored in a doubly linked list
- There is no synchronization in LinkedList, which provides efficient insertions and deletions. So the manipulation is faster because no shifting is required.
- The LinkedList class has all of the same methods as the ArrayList class because they both implement the List interface. This means that you can add items, change items, remove items and clear the list in the same way.



Set Interface

- A set is an interface and part of Collection that cannot have duplicate elements/values.
- It stands for an unordered set of elements
- Below classes are implemented Set interface:
 - HashSet
 - LinkedHashSet
 - TreeSet
- **Note:** HashSet doesn't maintain order; LinkedHashSet maintains as input order and TreeSet maintains ascending order.



HashSet

- HashSet is a class used to create a collection that uses a hash table or hashing mechanism for storage data. It implements Set interface. No index uses in HashSet.
- It cannot have duplicate values unlike ArrayList but allows null values.
- It doesn't maintain the insertion order like ArrayList.
- It is the best approach for search operations.
- Since there is no index number uses; so `get(index#)` method doesn't work to retrieve data.
- Use for-each loop or `iterator()` method to fetch data from the HashSet container.



LinkedHashSet <=>

- LinkedHashSet extends the HashSet class and implements the Set interface.
- Unlike a regular HashSet, which does not guarantee any specific order, a LinkedHashSet maintains the insertion order.
- It does not allow duplicate elements, similar to other Set implementations.
- It can store a null value as an element.



[linkedin.com/in/sarifuli/](https://www.linkedin.com/in/sarifuli/)

Queue Interface

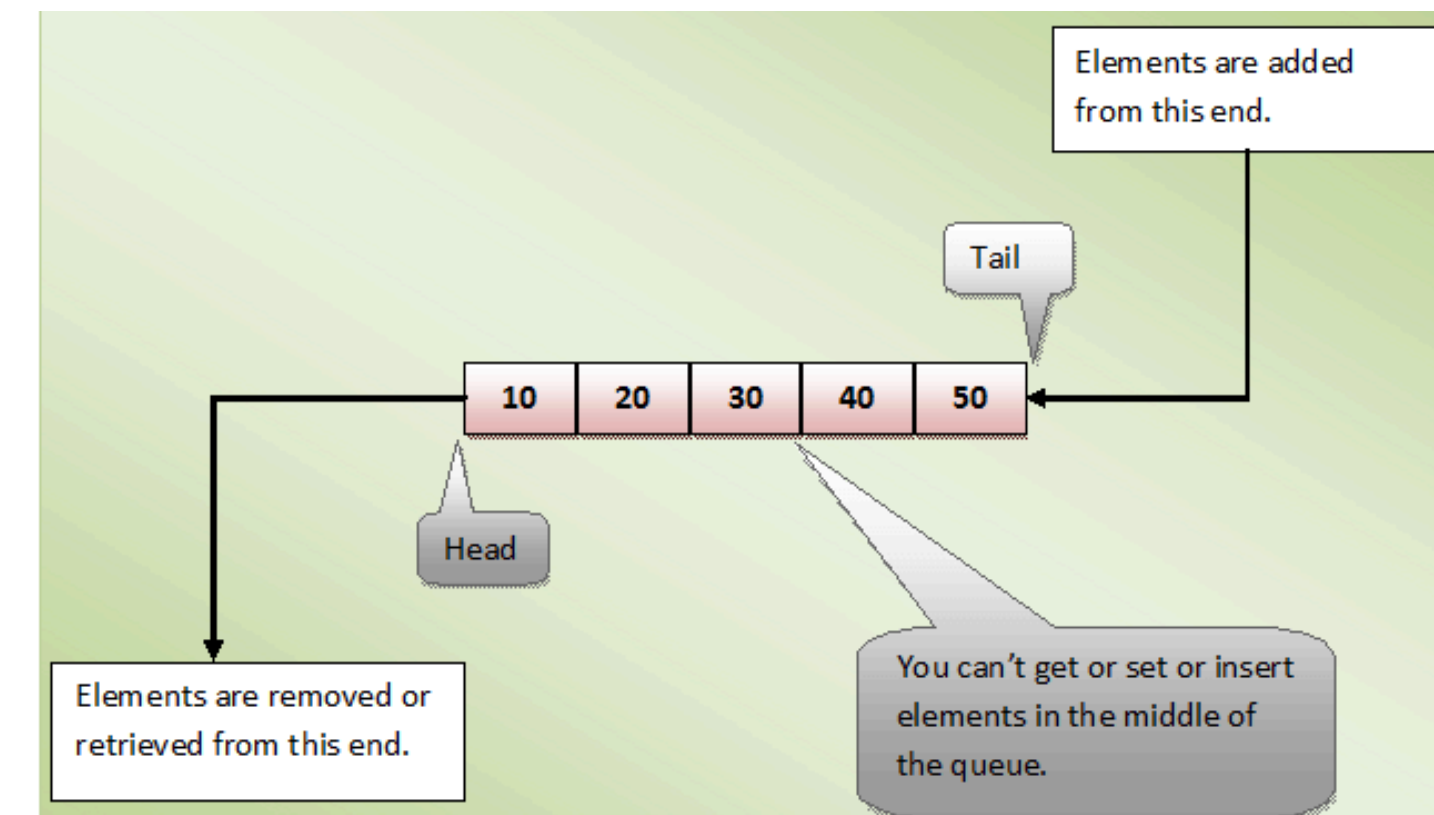
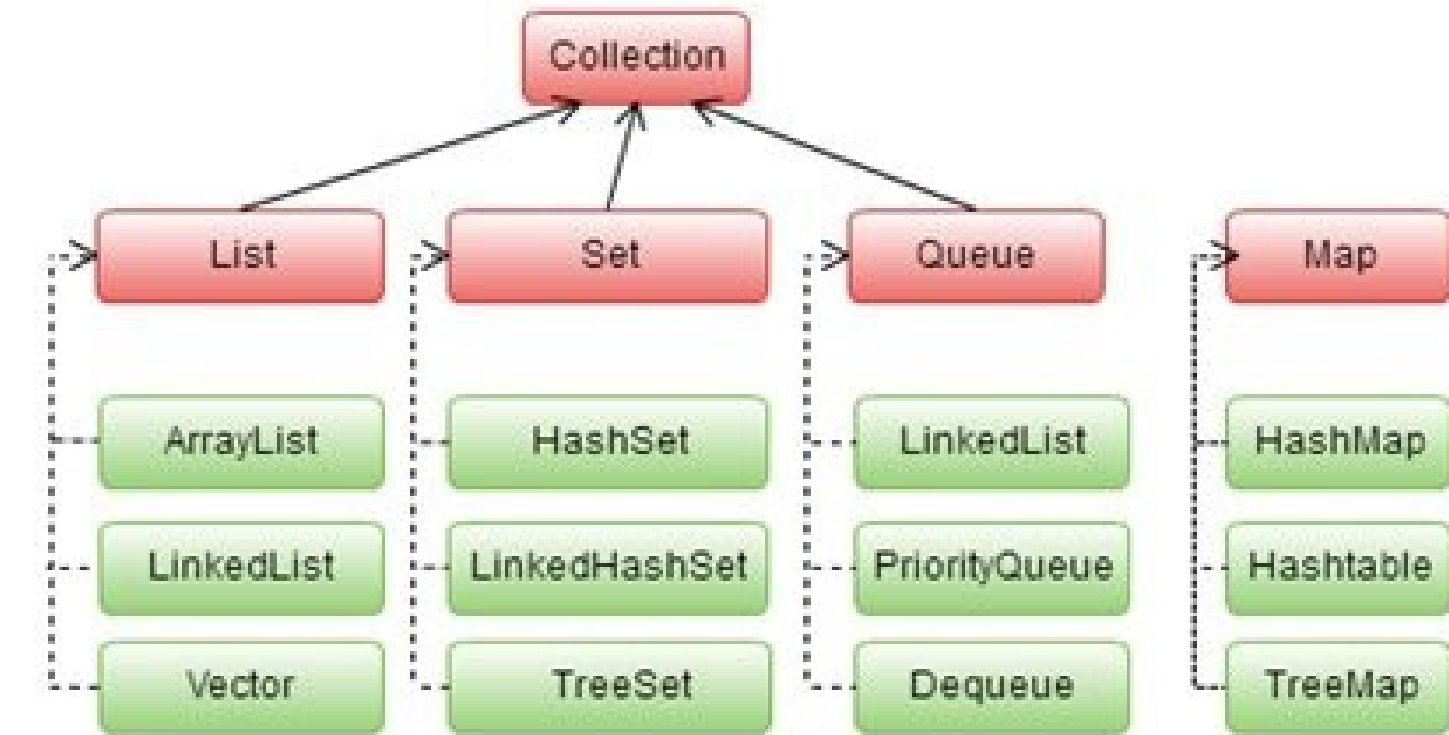
- The Queue interface represents a collection designed for holding elements prior to processing. It follows the First-In-First-Out (FIFO) principle
- Useful for processing tasks in the order they arrive.

- Bellow classes are implemented Queue interface:

- PriorityQueue
- LinkedList
- Dequeue

- Methods of Queue Interface:

Operation	Throws An Exception If operation is not possible	Returns null or false if operation is not possible
Add an element to the queue.	<code>add()</code>	<code>offer()</code>
Retrieve an element from the head of the queue.	<code>element()</code>	<code>peek()</code>
Retrieve And Remove an element from the head of the queue.	<code>remove()</code>	<code>poll()</code>

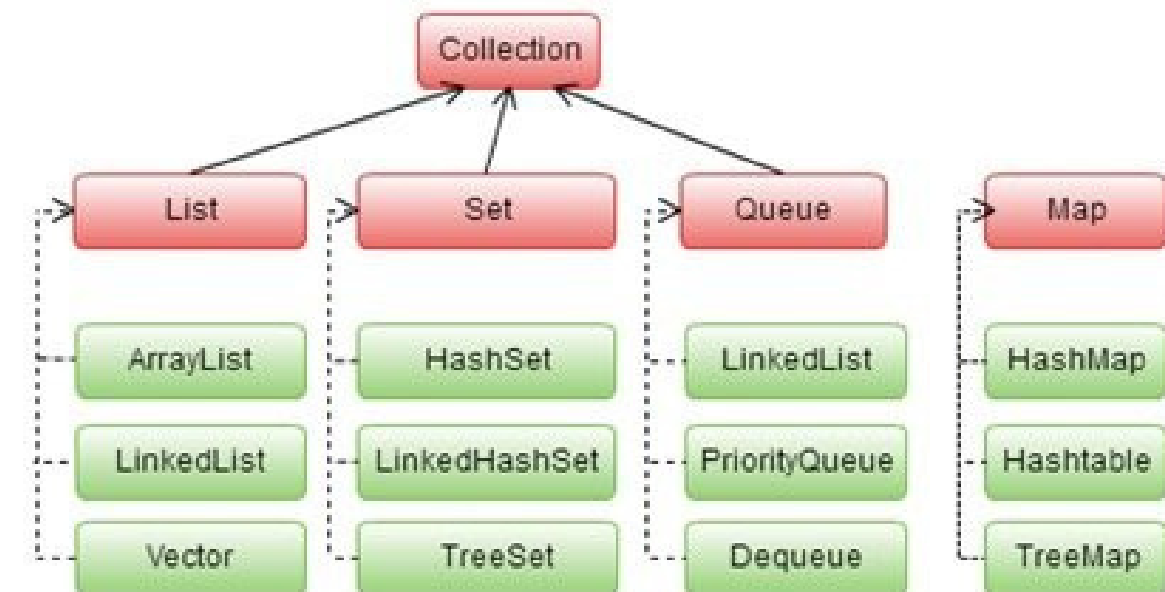


Map Interface

- It's a part of Collection framework but it doesn't implement the java.util.collection interface. Other than java.util.map interface
- It represents a collection of key-value pairs. So it stores pairs of Keys & values.
- Each key is linked to a specific value. Once stored in a map, we can look up the value just using the key
- A map cannot have duplicate keys but it can have duplicate values
- Bellow classes are implemented Map interface:
 - **HashMap**: Stores key-value pairs in a hash table. It does not maintain any order of keys. It allows one null key and multiple null value
 - **Hashtable**: Synchronized version of HashMap and does not allow null keys or values
 - **TreeMap**: Stores key-value pairs in a sorted order based on the keys, using a red-black tree. It does not allow null keys but allows multiple null values.
 - **LinkedHashMap**: Extends HashMap and maintains the insertion order of element

- **Common Methods in the Map Interface:**

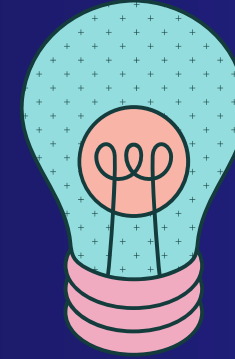
- put(K key, V value): Associates the specified value with the specified key
- putIfAbsent(K key, V value): Associates the specified value with the key if it's not already associated with a value.
- get(Object key): Returns the value associated with the specified key, or null if the key is not found.
- remove(Object key): Removes the mapping for the specified key if it exists.
- containsKey(Object key): Returns true if the map contains a mapping for the specified key.
- containsValue(Object value): Returns true if the map contains one or more keys mapped to the specified value.
- values(): Returns a Collection of all values contained in the map
- keySet(): Returns a Set of all the keys contained in the map
- entrySet(): Returns a Set of all the key-value mappings contained in the map.



[linkedin.com/in/sarifuli/](https://www.linkedin.com/in/sarifuli/)

Java { } Wrapper Classes

By <Sariful>



www.linkedin.com/in/sarifuli/



www.youtube.com/CodenboxAutomationLab



www.codenboxAutomationlab.com



Wrapper Classes

- Wrapper classes are those, which are used to 'wrap' primitive variables or data types (int, double, char, etc.) in such a way, so that they behave like an object.
- Java is an object-oriented language, and sometimes, objects are required instead of primitive types (e.g., in collections like ArrayList). So, Wrapper classes "wrap" primitive types within an object, which enable access to several utility methods.
- Key Wrapper classes (Each primitive type in Java has a corresponding wrapper class):
 - Integer—for wrapping int value.
 - Character—for wrapping char value.
 - Boolean—for wrapping boolean value.
 - Long—for wrapping long value.
 - Float—for wrapping float value.
 - Double—for wrapping double value



[linkedin.com/in/sarifuli/](https://www.linkedin.com/in/sarifuli/)



Wrapper Classes

- **Note** : Java supports autoboxing and unboxing, which simplifies conversions between primitives and their corresponding wrapper objects:
 - Wrapping: When a primitive value wrap to an object is called wrapping. In java Collection it's happened automatically; that's why wrapping is called 'Autoboxing'.
 - UnWrapping: When an object assign to a primitive value is called unwrapping or unboxing.
 - AutoBoxing: Autoboxing Is basically a technique in which compiler automatically wrap and unwrap the primitives in order to store them in "ArrayList"
- Useful Methods:
 - add()
 - get()
 - parsing strings to primitives (Integer.parseInt("123"))



[linkedin.com/in/sarifuli/](https://www.linkedin.com/in/sarifuli/)



THANK YOU 🙏

Get more Tips & Free tutorials 🚀 ▼



[linkedin.com/in/sarifuli/](https://www.linkedin.com/in/sarifuli/)



[youtube.com/@CodenboxAutomationLab](https://www.youtube.com/@CodenboxAutomationLab)



<https://codenboxautomationlab.com>



<https://x.com/CodenboxTeam>

